MAE 6291
Internet of Things for Engineers

Prof. Kartik Bulusu, MAE Dept.

Week 4 [02/11/2026]	• Setting up the Edge Lab • IoT Architecture and Ecosystem • Layers in IoT systems - 3 level layer model	• The Linux Terminal • Setting up a Python virtual environment [venv] - Sandboxing your code • Bash script to run RPi-python code • Blinking LEDs on boot [Graded lab activity]

git clone https://github.com/gwu-mae6291-iot/spring2026_codes.git

GW School of Engineering & Applied Science
THE GEORGE WASHINGTON UNIVERSITY
Spring 2026

Photo: Kartik Bulusu

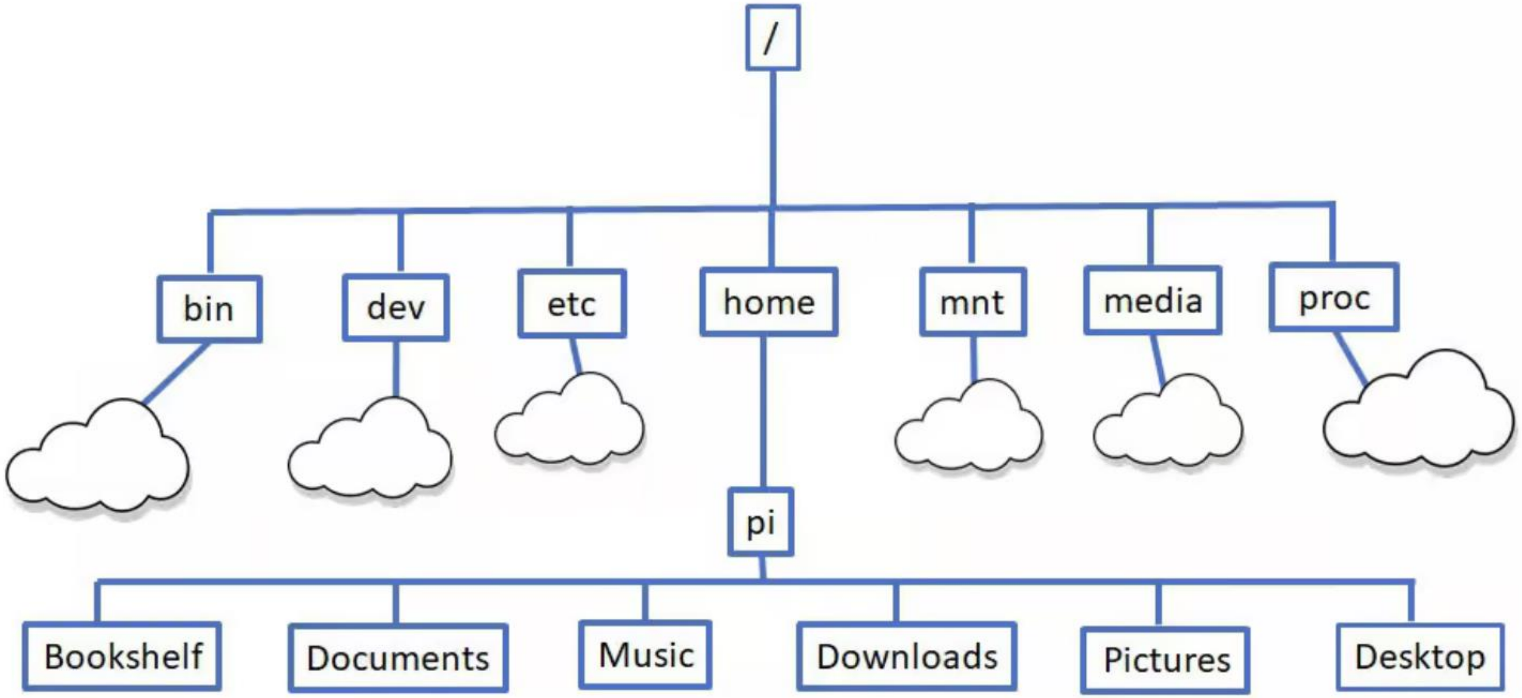
Content Attribution & Sources

This course material incorporates both original content and supplementary visual aids:

- **Images with references:** Properly cited in slide captions, figure legends, or in-text attributions
- **Unreferenced images:** Sourced from Perplexity.io and used for illustrative purposes in an educational context
- **Fair use:** All materials are used in compliance with educational fair use guidelines

The Linux Terminal

Source:
https://www.circuitbasics.com/how-to-navigate-the-raspberry-pi-file-system/



Explore files and folders

Command	Description
pwd	Show the **current** directory you are in
ls	List files in the current directory; add -l or -a for more detail or to see hidden files
cd /path/goes/here	Change directory (for example cd /home/pi/Documents)
mkdir myfolder	Create a new directory named myfolder
cp file1 file2	Copy a file; cp -r dir1 dir2 copies a whole directory
mv old new	Move or rename files/directories
rm file	Delete a file; use rm -r folder carefully to remove a directory

Helpful navigation and history tips

Command or Keyboard Entry	Description
clear	Clear the terminal screen.
history	Show a numbered list of past commands; run one again with !number.
Up/Down arrow keys	Scroll through previously used commands to run or edit them.

View and edit files

Command	Description
nano file.txt	Open a simple text editor in the terminal to create or edit a file
less file.txt	View a long file one page at a time (use q to quit)
cat file.txt	Print the contents of a text file to the screen

System information and status

Command	Description
hostname -I	Show the Pi's IP address
uname -a	Show Linux kernel and system details
df -h	Show disk space usage in human-readable form
free -h	Show memory (RAM) usage
uptime	See how long the Pi has been running
top or htop	Monitor running processes and CPU usage (press q to exit top).

IP address of your RPi connected to the network using WiFi

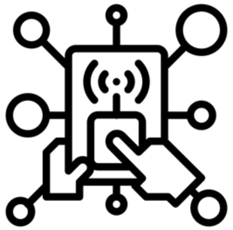
ifconfig -a

```
pi@raspberrypi017: ~
File  Edit  Tabs  Help
pi@raspberrypi017:~ $ ifconfig -a
docker0: flags=4099<UP,BROADCAST,MULTICAST>  mtu 1500
        inet 172.17.0.1  netmask 255.255.0.0  broadcast 172.17.255.255
        ether ea:e0:b9:19:85:8f  txqueuelen 0  (Ethernet)
        RX packets 0  bytes 0 (0.0 B)
        RX errors 0  dropped 0  overruns 0  frame 0
        TX packets 0  bytes 0 (0.0 B)
        TX errors 0  dropped 0 overruns 0  carrier 0  collisions 0

eth0: flags=4163<UP,BROADCAST,RUNNING,MULTICAST>  mtu 1500
        inet 169.254.247.201  netmask 255.255.0.0  broadcast 169.254.255.255
        inet6 fe80::21e4:d41c:791a:a9d7  prefixlen 64  scopeid 0x20<link>
        ether dc:a6:32:98:38:24  txqueuelen 1000  (Ethernet)
        RX packets 1874  bytes 166519 (162.6 KiB)
        RX errors 0  dropped 0  overruns 0  frame 0
        TX packets 1723  bytes 1258105 (1.1 MiB)
        TX errors 0  dropped 0 overruns 0  carrier 0  collisions 0

lo: flags=73<UP,LOOPBACK,RUNNING>  mtu 65536
        inet 127.0.0.1  netmask 255.0.0.0
        inet6 ::1  prefixlen 128  scopeid 0x10<host>
        loop  txqueuelen 1000  (Local Loopback)
        RX packets 53  bytes 5297 (5.1 KiB)
        RX errors 0  dropped 0  overruns 0  frame 0
        TX packets 53  bytes 5297 (5.1 KiB)
        TX errors 0  dropped 0 overruns 0  carrier 0  collisions 0

wlan0: flags=4163<UP,BROADCAST,RUNNING,MULTICAST>  mtu 1500
        inet 10.198.24.23  netmask 255.255.252.0  broadcast 10.198.27.255
        inet6 fe80::3d8a:ccb:6a3a:2f0  prefixlen 64  scopeid 0x20<link>
        ether dc:a6:32:98:38:25  txqueuelen 1000  (Ethernet)
        RX packets 12  bytes 1379 (1.3 KiB)
        RX errors 0  dropped 0  overruns 0  frame 0
        TX packets 67  bytes 8066 (7.8 KiB)
        TX errors 0  dropped 0 overruns 0  carrier 0  collisions 0

pi@raspberrypi017:~ $
```

Building up the IoT Architecture and Ecosystem

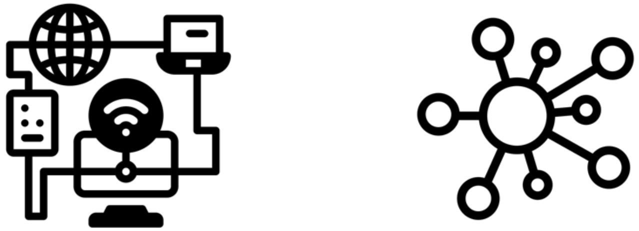
Icon Sources:
sensor by Carolina Cani:, sensor by Pham Duy Phuong Hung, sensor by Tippawan Sookruay, sensor by Lorenzo: https://thenounproject.com/browse/icons/term/sensor
fire sensor by LAFS : https://thenounproject.com/browse/icons/term/fire-sensor/
Ultrasound by Shocho: https://thenounproject.com/browse/icons/term/ultrasound/
Network by Solikin:, Network by Tippawan: https://thenounproject.com/browse/icons/term/network
application by Chaowalit Koetchuea: https://thenounproject.com/browse/icons/term/application/

Information-layer



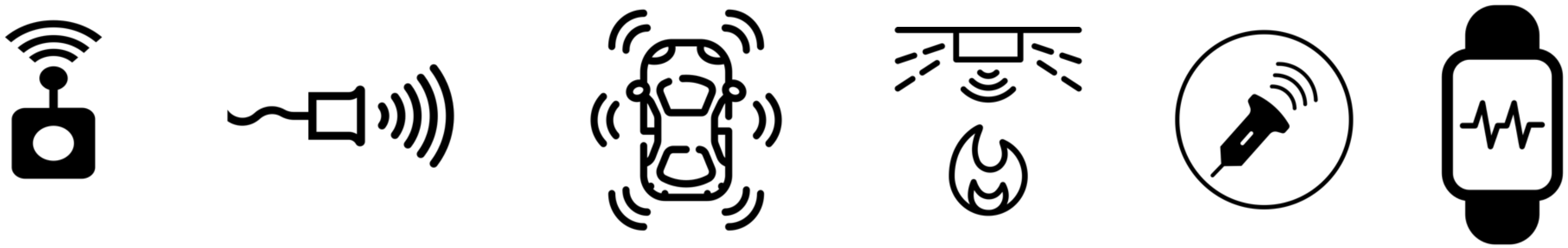
Data

Communication-layer



Connectivity

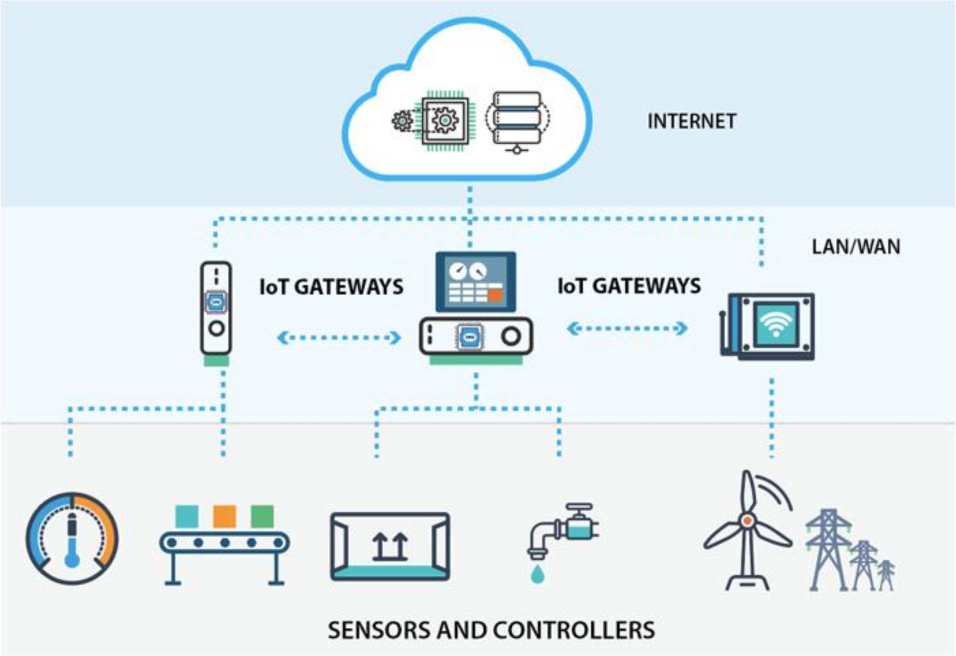
Sensor-layer

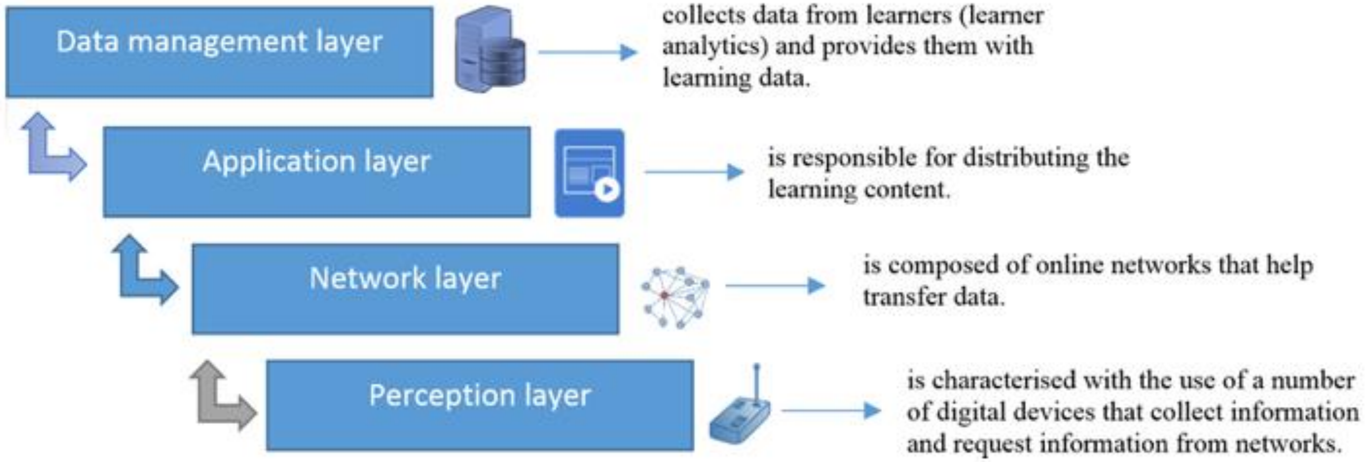


Things

Sources:
Internet of Things, IoT Architecture: https://www.zibtek.com/blog/iot-architecture/

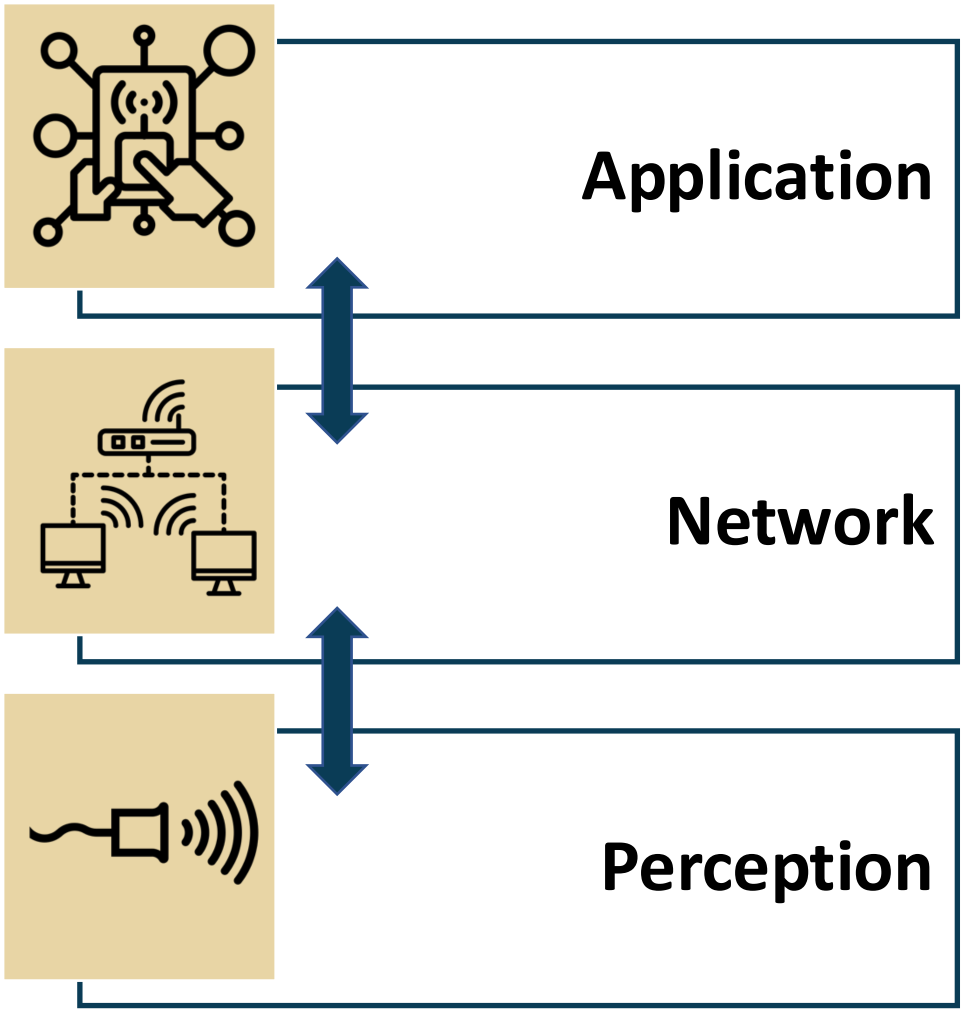
Refining and defining the 3-Layer IoT Architecture





The 3-Layer IoT Architecture

Icon Sources:
sensor by Carolina Cani:, sensor by Pham Duy Phuong Hung, sensor by Tippawan Sookruay, sensor by Lorenzo: https://thenounproject.com/browse/icons/term/sensor
wifi network by Matthias Hartmann:: https://thenounproject.com/browse/icons/term/wifi-network/
application by Chaowalit Koetchuea: https://thenounproject.com/browse/icons/term/application/



Skeleton of the updated Python program written for Raspberry Pi

```
import library1 as name1
import RPi.GPIO as GPIO
import time
```

Import libraries that are relevant for interaction with the Raspberry Pi hardware such as GPIO pins, camera ports etc.

```
GPIO.setmode(GPIO.BOARD)

GPIO.setup(12, GPIO.OUT)
```

Set up GPIO pins as data outputs or inputs for sensors and actuators

```
def function_name(arguments):
        statement
        statement
        . . .
        return value
```

Create user-defined functions to modularize your code and make it easy to work.

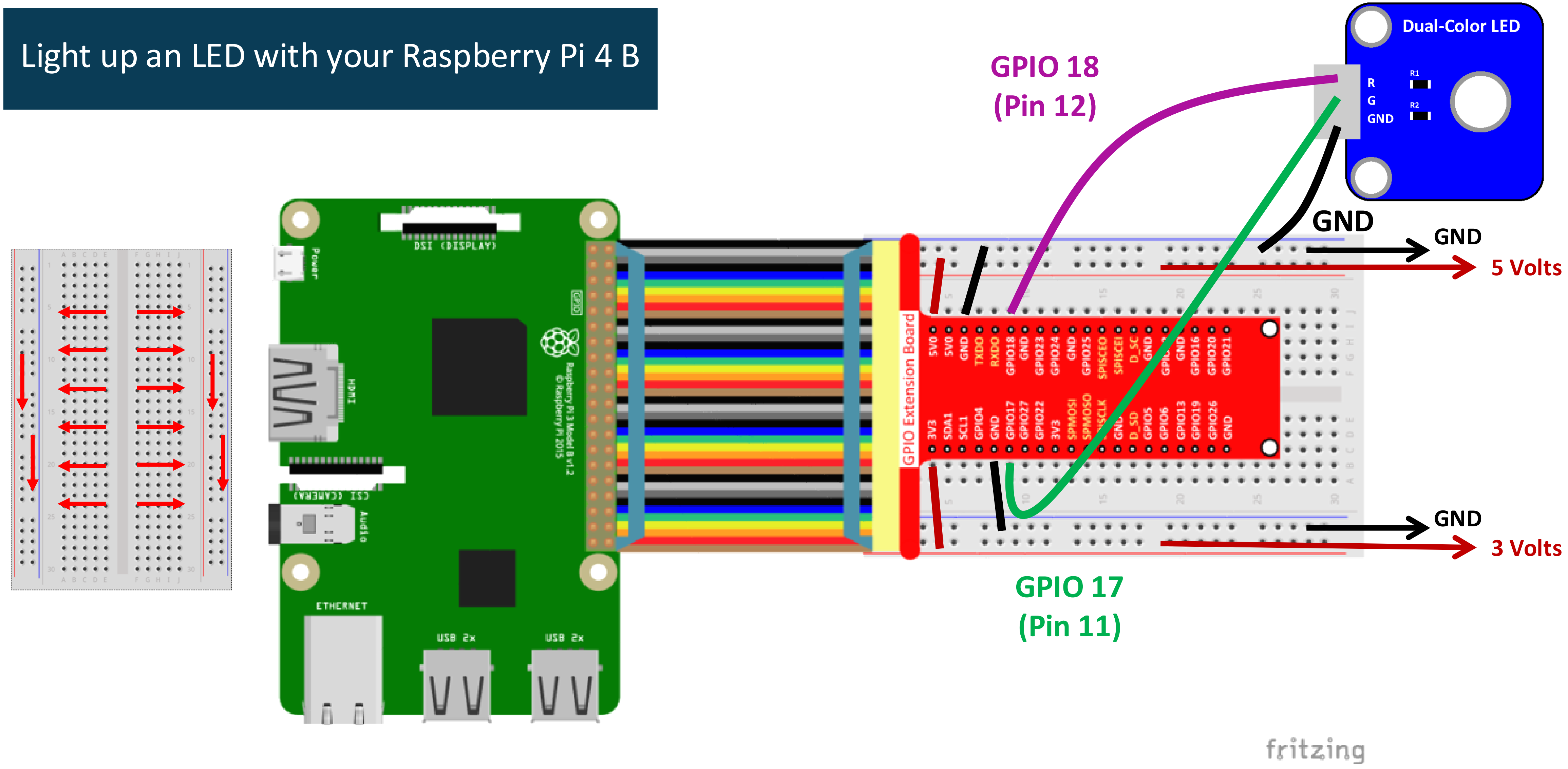
Create user-defined functions to release the GPIO pins

```
if __name__ == "__main__":
        try:
                function_name1()
        except KeyboardInterrupt:
                function_name2()
```

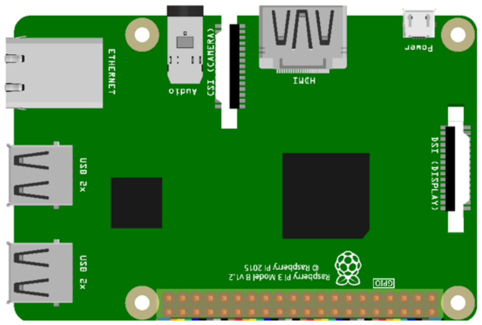
Create entry point into the program and pull all functions in

Create a keyboard-interrupt exception clause

Light up an LED with your Raspberry Pi 4 B

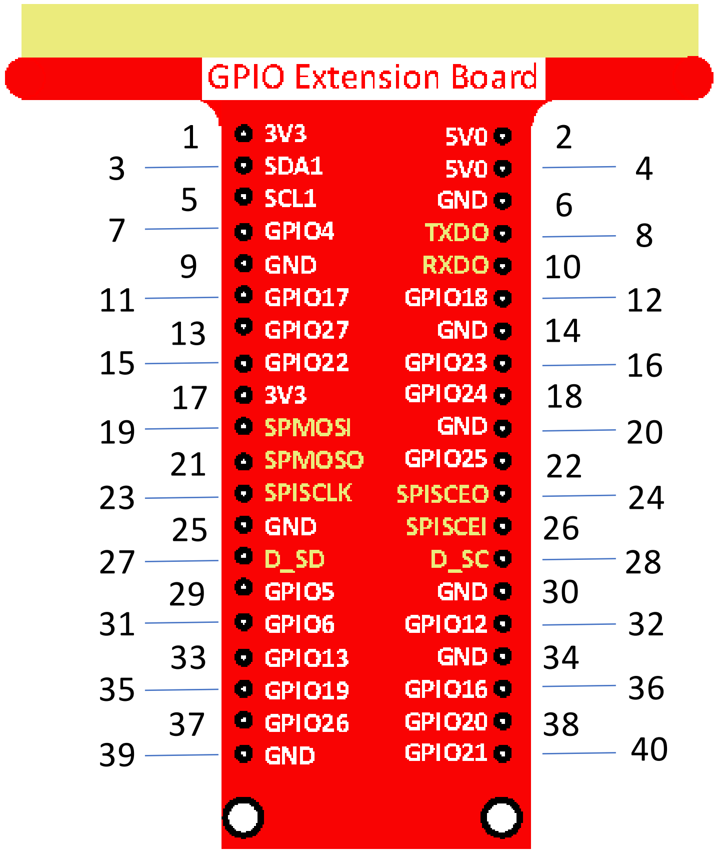


Another python code to kick start your Raspberry Pi Model 3 B+ (RPi)



```
import RPi.GPIO as GPIO
import time
```

```
GPIO.setmode(GPIO.BOARD)
```




```
GPIO.setup(12, GPIO.OUT)
```

```
def loop():
        while True:
                GPIO.output(12, GPIO.HIGH)
        time.sleep(0.5)
                GPIO.output(12, GPIO.LOW)
        time.sleep(0.5)
```

```
def destroy():
    GPIO.output(12, GPIO.LOW)
            # Turn off all leds
    GPIO.cleanup()
```

```
if __name__ == "__main__":
    try:
        loop()
    except KeyboardInterrupt:
        destroy()
```

Setting up a Python virtual environment - "Sandbox" your Python projects

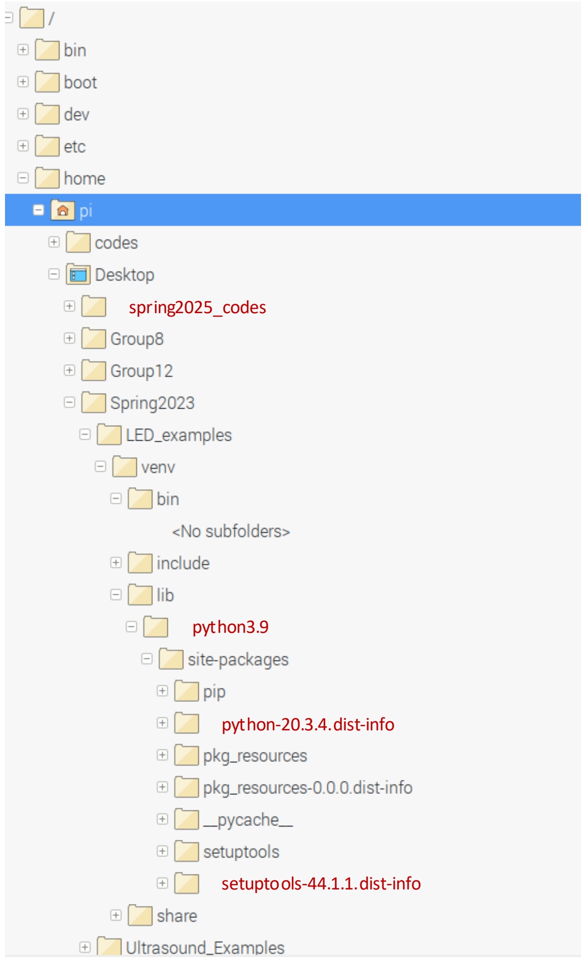
"Unwearyingly working and making changes to the global Python environment can break Python-based system-level tools, and remedying the situation can become a major headache."
Gary Smart, Practical Python Programming for IoT

Create python virtual environments

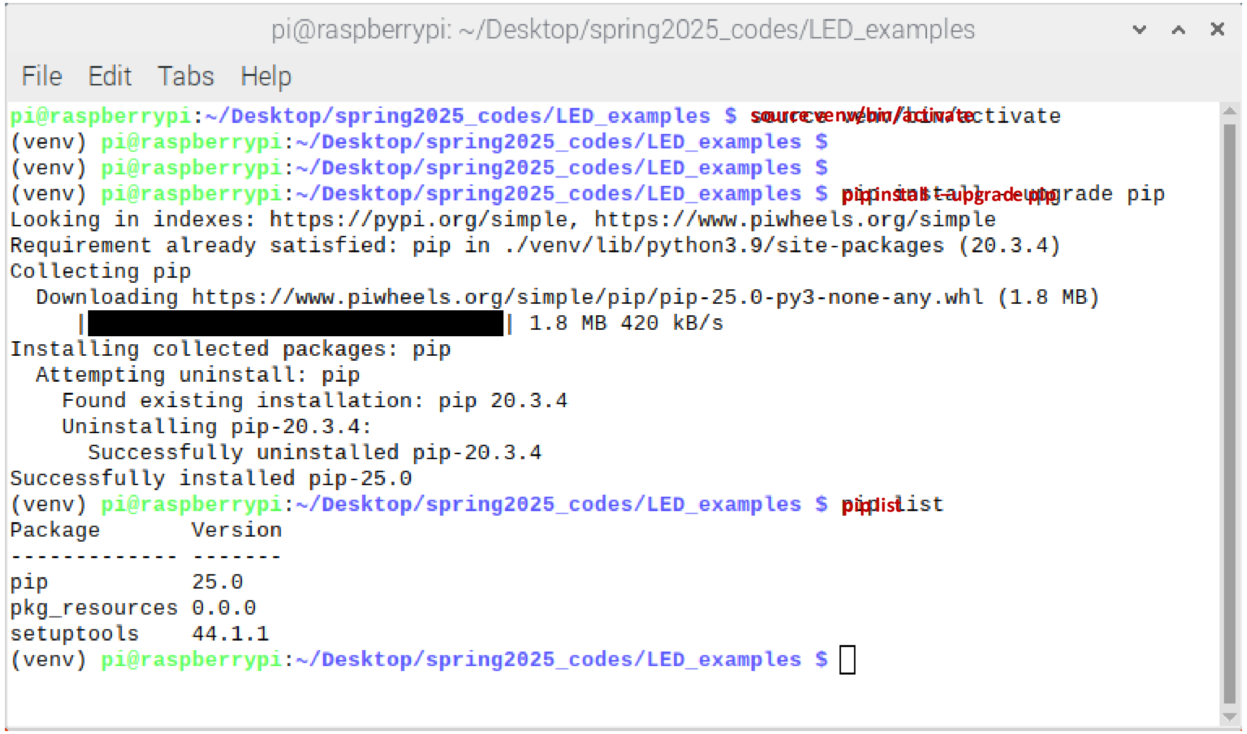
- To "sandbox" our Python projects
- Avoid adverse interference with
  - system-level Python utilities
  - or other Python projects.

```
pi@raspberrypi: ~/Desktop/spring2025_codes/LED_examples
File  Edit  Tabs  Help
pi@raspberrypi:~ $ cd Desktop/spring2025_codes/LED_examples/
pi@raspberrypi:~/Desktop/spring2025_codes/LED_examples $
pi@raspberrypi:~/Desktop/spring2025_codes/LED_examples $
pi@raspberrypi:~/Desktop/spring2025_codes/LED_examples $ python -m venv venv
pi@raspberrypi:~/Desktop/spring2025_codes/LED_examples $
pi@raspberrypi:~/Desktop/spring2025_codes/LED_examples $
pi@raspberrypi:~/Desktop/spring2025_codes/LED_examples $ source venv/bin/activate
(venv) pi@raspberrypi:~/Desktop/spring2025_codes/LED_examples $
(venv) pi@raspberrypi:~/Desktop/spring2025_codes/LED_examples $
(venv) pi@raspberrypi:~/Desktop/spring2025_codes/LED_examples $ which python
/home/pi/Desktop/spring2025_codes/LED_examples/venv/bin/python
(venv) pi@raspberrypi:~/Desktop/spring2025_codes/LED_examples $
(venv) pi@raspberrypi:~/Desktop/spring2025_codes/LED_examples $
(venv) pi@raspberrypi:~/Desktop/spring2025_codes/LED_examples $ python --version
Python 3.9.2
(venv) pi@raspberrypi:~/Desktop/spring2025_codes/LED_examples $
(venv) pi@raspberrypi:~/Desktop/spring2025_codes/LED_examples $
(venv) pi@raspberrypi:~/Desktop/spring2025_codes/LED_examples $ deactivate
pi@raspberrypi:~/Desktop/spring2025_codes/LED_examples $
pi@raspberrypi:~/Desktop/spring2025_codes/LED_examples $
pi@raspberrypi:~/Desktop/spring2025_codes/LED_examples $
```

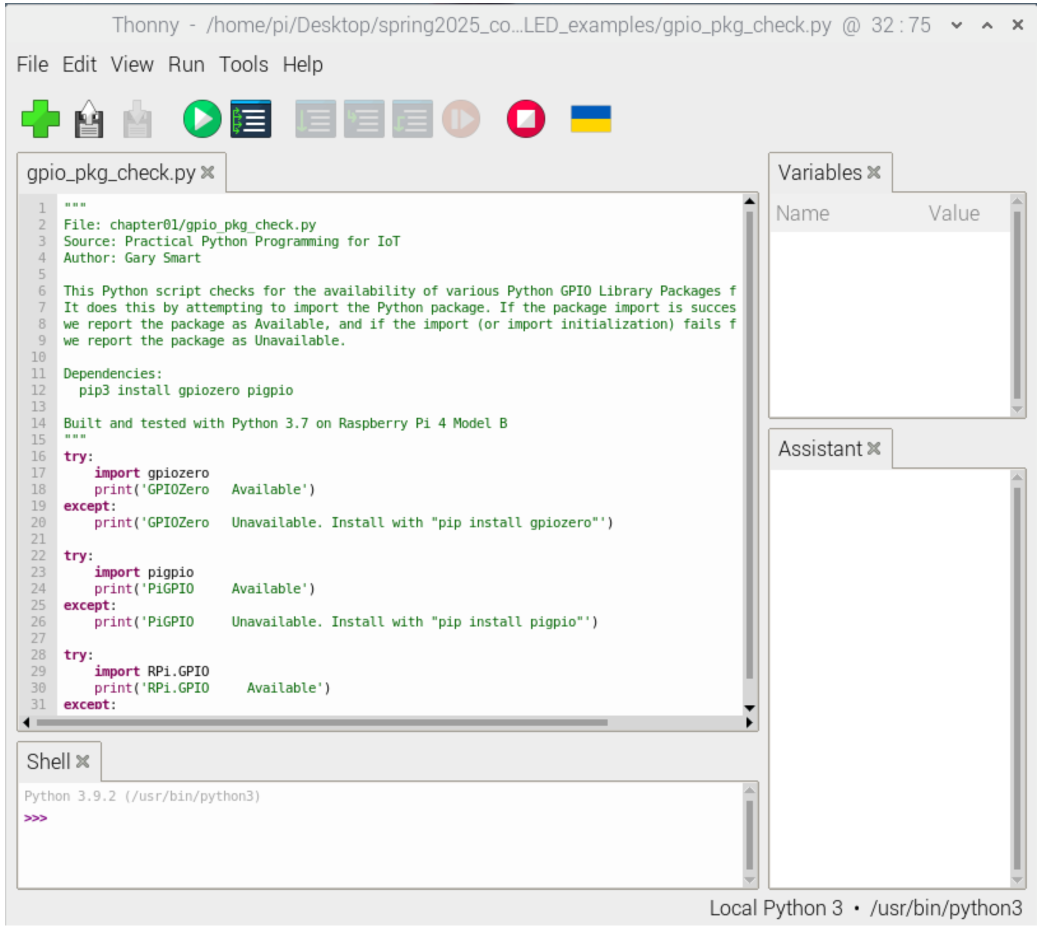
Anatomy of the python virtual environments



Activate python-venv
&
Install pip



Review of the code provided to you



Install GPIO packages using the code provided.
Note: You are still in python-venv

Checks and balances:
pip list
pip freeze > requirements

```
pi@raspberrypi: ~/Desktop/spring2025_codes/LED_examples
File  Edit  Tabs  Help
(venv) pi@raspberrypi:~/Desktop/spring2025_codes/LED_examples $ python gpio_pkg_check.py
GPIOZero    Unavailable. Install with "pip install gpiozero"
PiGPIO      Unavailable. Install with "pip install pigpio"
(venv) pi@raspberrypi:~/Desktop/spring2025_codes/LED_examples $ pip install gpiozero pigpio RPi.GPIO
Looking in indexes: https://pypi.org/simple, https://www.piwheels.org/simple
Collecting gpiozero
  Downloading https://www.piwheels.org/simple/gpiozero/gpiozero-2.0.1-py3-none-any.whl (150 kB)
Collecting pigpio
  Downloading https://www.piwheels.org/simple/pigpio/pigpio-1.78-py2.py3-none-any.whl (39 kB)
Collecting RPi.GPIO
  Downloading https://www.piwheels.org/simple/rpi-gpio/RPi.GPIO-0.7.1-cp39-cp39-linux_armv7l.whl (66 kB)
Collecting colorzero (from gpiozero)
  Downloading https://www.piwheels.org/simple/colorzero/colorzero-2.0-py2.py3-none-any.whl (26 kB)
Collecting importlib-resources~=5.0 (from gpiozero)
  Downloading https://www.piwheels.org/simple/importlib-resources/importlib_resources-5.13.0-py3-none-any.whl (32 kB)
Collecting importlib-metadata~=4.6 (from gpiozero)
  Downloading https://www.piwheels.org/simple/importlib-metadata/importlib_metadata-4.13.0-py3-none-any.whl (23 kB)
Collecting zipp>=0.5 (from importlib-metadata~=4.6->gpiozero)
  Downloading https://www.piwheels.org/simple/zipp/zipp-3.21.0-py3-none-any.whl (9.6 kB)
Requirement already satisfied: setuptools in ./venv/lib/python3.9/site-packages (from colorzero->gpiozero) (44.1.1)
Installing collected packages: RPi.GPIO, pigpio, zipp, colorzero, importlib-resources, importlib-metadata, gpiozero
Successfully installed RPi.GPIO-0.7.1 colorzero-2.0 gpiozero-2.0.1 importlib-metadata-4.13.0 importlib-resources-5.13.0 pigp
io-1.78 zipp-3.21.0
(venv) pi@raspberrypi:~/Desktop/spring2025_codes/LED_examples $
```

```
pi@raspberrypi: ~/Desktop/spring2025_codes/LED_examples
File  Edit  Tabs  Help
(venv) pi@raspberrypi:~/Desktop/spring2025_codes/LED_examples $ pip list
Package             Version
------------------- -------
colorzero           2.0
gpiozero            2.0.1
importlib-metadata  4.13.0
importlib-resources 5.13.0
pigpio              1.78
pip                 25.0
pkg_resources       0.0.0
RPi.GPIO            0.7.1
setuptools          44.1.1
zipp                3.21.0
(venv) pi@raspberrypi:~/Desktop/spring2025_codes/LED_examples $ python gpio_pkg_check.py
GPIOZero    Available
PiGPIO      Available
RPi.GPIO      Available
(venv) pi@raspberrypi:~/Desktop/spring2025_codes/LED_examples $ pip freeze > requirements
(venv) pi@raspberrypi:~/Desktop/spring2025_codes/LED_examples $
(venv) pi@raspberrypi:~/Desktop/spring2025_codes/LED_examples $
```

pip stands for "Python installs packages".
pip queries the Python Package Index, or simply PyPi
Web source for PyPi: https://pypi.org

Run the python codes
Blink your LEDs
Note: You are still in python-venv

deactivate your venv

```
pi@raspberrypi: ~/Desktop/spring2025_codes/LED_examples
File  Edit  Tabs  Help
(venv) pi@raspberrypi:~/Desktop/spring2025_codes/LED_examples $ which python
/home/pi/Desktop/spring2025_codes/LED_examples/venv/bin/python
(venv) pi@raspberrypi:~/Desktop/spring2025_codes/LED_examples $
(venv) pi@raspberrypi:~/Desktop/spring2025_codes/LED_examples $
(venv) pi@raspberrypi:~/Desktop/spring2025_codes/LED_examples $ python --version
Python 3.9.2
(venv) pi@raspberrypi:~/Desktop/spring2025_codes/LED_examples $
(venv) pi@raspberrypi:~/Desktop/spring2025_codes/LED_examples $
(venv) pi@raspberrypi:~/Desktop/spring2025_codes/LED_examples $ python gpio_pkg_check.py
GPIOZero   Available
PiGPIO     Available
RPi.GPIO     Available
(venv) pi@raspberrypi:~/Desktop/spring2025_codes/LED_examples $ python LED_1.py
0
1
2
3
4
(venv) pi@raspberrypi:~/Desktop/spring2025_codes/LED_examples $ clear
```

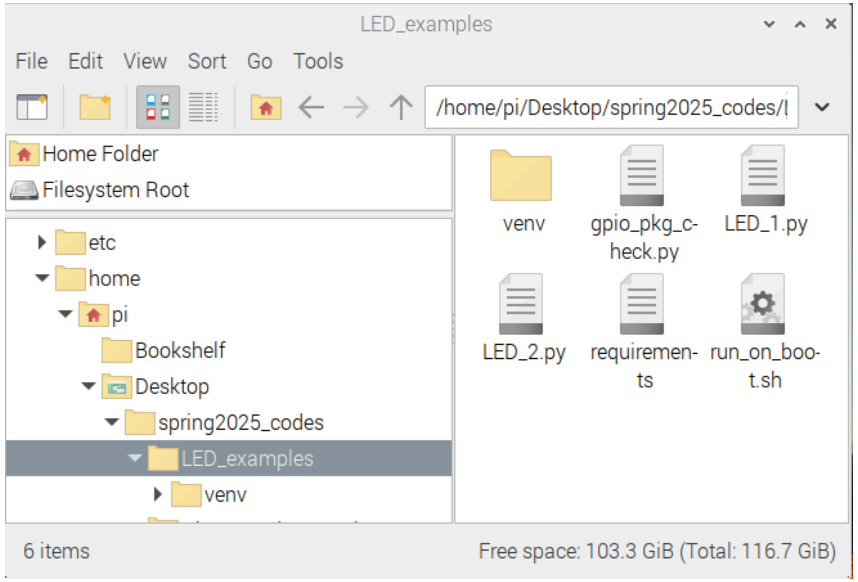
```
pi@raspberrypi: ~/Desktop/spring2025_codes/LED_examples
File  Edit  Tabs  Help
(venv) pi@raspberrypi:~/Desktop/spring2025_codes/LED_examples $ deactivate
pi@raspberrypi:~/Desktop/spring2025_codes/LED_examples $
pi@raspberrypi:~/Desktop/spring2025_codes/LED_examples $
pi@raspberrypi:~/Desktop/spring2025_codes/LED_examples $
pi@raspberrypi:~/Desktop/spring2025_codes/LED_examples $
pi@raspberrypi:~/Desktop/spring2025_codes/LED_examples $
```

Running a python script at boot
cron, the UNIX scheduler

Awesome IoT project will run automatically every time you start your Raspberry Pi

Review run_on_boot.sh
Note: You are no longer in python-venv

Let's make sure we have a file-folder structure that looks some what like this



Let's open the *run_on_boot.sh* *on a terminal* to review the script

```
pi@raspberrypi: ~/Desktop/spring2025_codes/LED_examples
File Edit Tabs Help
pi@raspberrypi:~/Desktop/spring2025_codes/LED_examples $ cat run_on_boot.sh
#!/bin/bash
#File: LED_examples/run_on_boot.sh

# Absolute path to Virtual Environment python interpreter
PYTHON=/home/pi/Desktop/spring2025_codes/LED_examples/venv/bin/python

# Absolute path to Python script
SCRIPT=//home/pi/Desktop/spring2025_codes/LED_examples/LED_1.py

# Absolute path to output log file
LOG=/home/pi/Desktop/spring2025_codes/LED_examples/LED_1.log

echo -e "\n####### STARTUP $(date) ######\n" >> $LOG

$PYTHON $SCRIPT >> $LOG 2>&1
pi@raspberrypi:~/Desktop/spring2025_codes/LED_examples $
pi@raspberrypi:~/Desktop/spring2025_codes/LED_examples $
```

Absolute paths to

1. Python interpreter in venv
2. Python script
3. Locate the LOG-file

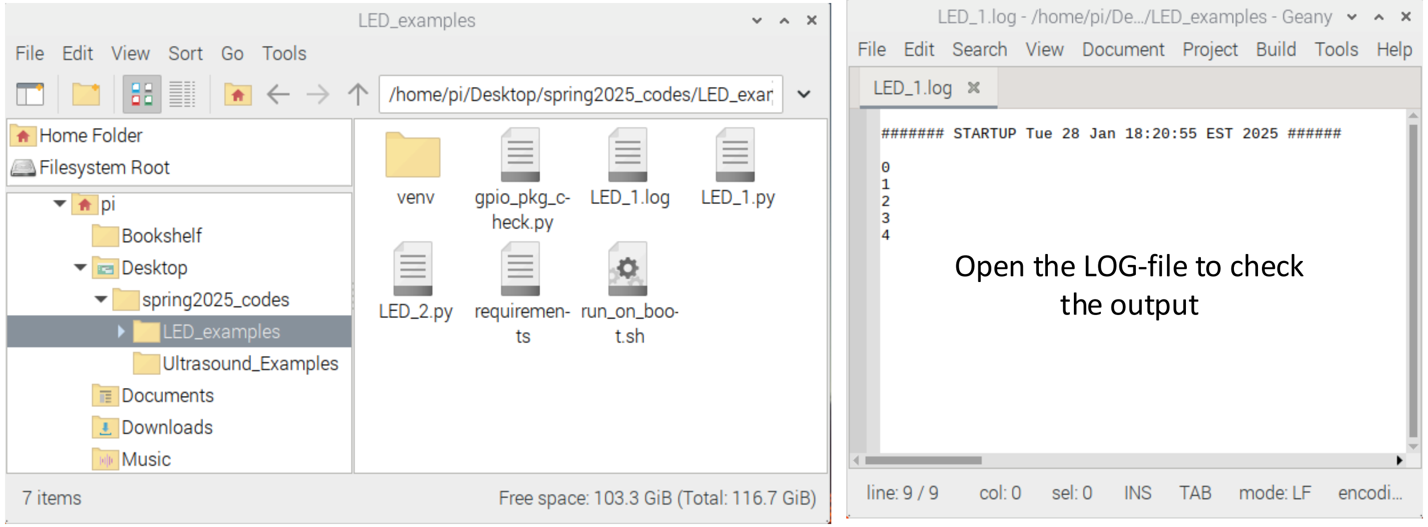
The last two lines tie everything together

1. echo [updates LOG-file with string in quotes]
2. Next line runs py-file and logs output on boot
3. 2>&1 [logs any additional errors during run]

chmod u+x run_on_boot.sh

*current **U**ser, make the file e**X**ecutable*

```
pi@raspberrypi:~/Desktop/spring2025_codes/LED_examples $ cat run_on_boot.sh
#!/bin/bash
#File: LED_examples/run_on_boot.sh

# Absolute path to Virtual Environment python interpreter
PYTHON=/home/pi/Desktop/spring2025_codes/LED_examples/venv/bin/python

# Absolute path to Python script
SCRIPT=//home/pi/Desktop/spring2025_codes/LED_examples/LED_1.py

# Absolute path to output log file
LOG=/home/pi/Desktop/spring2025_codes/LED_examples/LED_1.log

echo -e "\n####### STARTUP $(date) ######\n" >> $LOG

$PYTHON $SCRIPT >> $LOG 2>&1
pi@raspberrypi:~/Desktop/spring2025_codes/LED_examples $
pi@raspberrypi:~/Desktop/spring2025_codes/LED_examples $ chmod u+x run_on_boot.sh
pi@raspberrypi:~/Desktop/spring2025_codes/LED_examples $
```



Open the LOG-file to check the output

./run_on_boot.sh

```
pi@raspberrypi:~/Desktop/spring2025_codes/LED_examples $
pi@raspberrypi:~/Desktop/spring2025_codes/LED_examples $ ./run_on_boot.sh
pi@raspberrypi:~/Desktop/spring2025_codes/LED_examples $
pi@raspberrypi:~/Desktop/spring2025_codes/LED_examples $
```

crontab -e

$ crontab –e

This is a list of editors used in terminal with Linux or Unix OS

I recommend you choose "nano" or the default Option-1 and hit "Enter".

"vim" and "ed" are also popular with advanced or legacy users

"code" will open up VS Code that slows down the Raspberry Pi OS

no crontab for pi - using an empty one
Select an editor. To change later, run 'select-editor'.

1. /bin/nano <---- easiest
2. /usr/bin/vim.tiny
3. /usr/bin/code
4. /bin/ed

Choose 1-4 [1]: 1

crontab -e

"Do not forget the & character at the end of the line. This makes sure the script runs in the background."

Gary Smart, Practical Python Programming for IoT

$ crontab –e

@reboot /home/pi/Desktop/spring2026_codes/LED_examples/run_on_boot.sh &

```
pi@raspberrypi: ~/Desktop/spring2025_codes/LED_examples
File  Edit  Tabs  Help
 GNU nano 5.4                  /tmp/crontab.MInc7f/crontab *
# Each task to run has to be defined through a single line
# indicating with different fields when the task will be run
# and what command to run for the task
#
# To define the time you can provide concrete values for
# minute (m), hour (h), day of month (dom), month (mon),
# and day of week (dow) or use '*' in these fields (for 'any').
#
# Notice that tasks will be started based on the cron's system
# daemon's notion of time and timezones.
#
# Output of the crontab jobs (including errors) is sent through
# email to the user the crontab file belongs to (unless redirected).
#
# For example, you can run a backup of all your user accounts
# at 5 a.m every week with:
# 0 5 * * 1 tar -zcf /var/backups/home.tgz /home/
#
# For more information see the manual pages of crontab(5) and cron(8)
#
# m h  dom mon dow   command

@reboot /home/pi/Desktop/spring2025_codes/LED_examples/run_on_boot.sh &

^G Help       ^O Write Out  ^W Where Is   ^K Cut        ^T Execute    ^C Location
^X Exit       ^R Read File  ^\ Replace    ^U Paste      ^J Justify    ^_ Go To Line
```

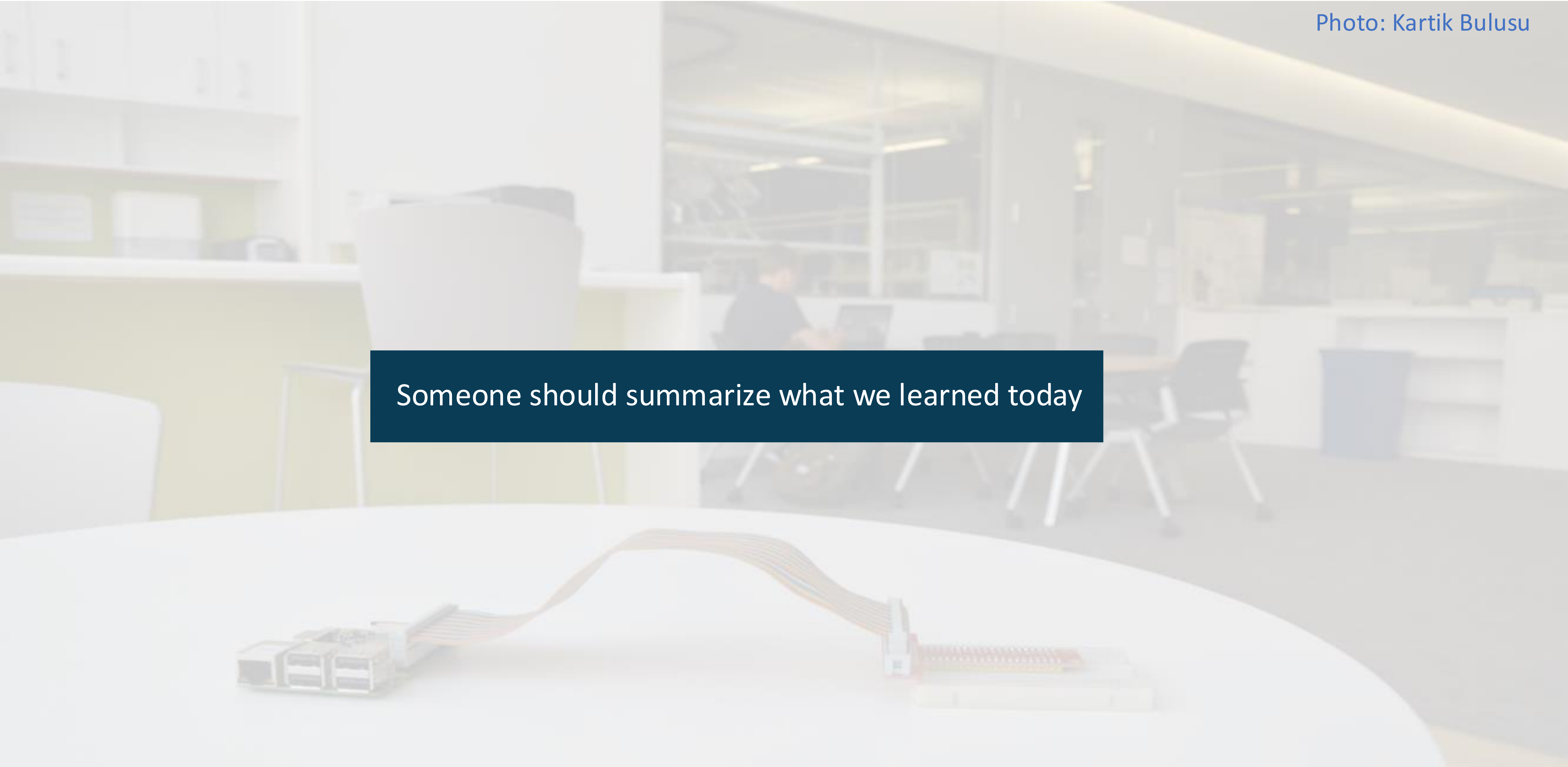
Try manually ./run_on_boot.sh
then
sudo reboot

```
pi@raspberrypi:~/Desktop/spring2025_codes/LED_examples $
pi@raspberrypi:~/Desktop/spring2025_codes/LED_examples $ ./run_on_boot.sh
pi@raspberrypi:~/Desktop/spring2025_codes/LED_examples $
pi@raspberrypi:~/Desktop/spring2025_codes/LED_examples $ sudo reboot
```

To stop a cron job you can type this on the terminal:

sudo service cron stop

Photo: Kartik Bulusu



Someone should summarize what we learned today